

Python 程序设计

第 6 讲 (b): Python 类的继承: Inheritance

HY Deng

hydeng@shiep.edu.cn



本讲的主要内容

1 1. Python 类的继承

- 1.1 类的继承基础知识
- 1.2 继承示例: Vector 类与子类

2 2. 案例学习

本讲的主要内容

1 1. Python 类的继承

■ 1.1 类的继承基础知识

■ 1.2 继承示例: Vector 类与子类

2 2. 案例学习

灾难案例: 重复定义的种类

Class 1: 学生类

```
class Student:
    def __init__(self, name, sid):
        self.name = name
        self.sid = sid

    def introduce(self):
        print(self.name, self.sid)
```

灾难案例: 重复定义的类

Class 2: 研究生类

```
class GraduateStudent:
    def __init__(self, name, sid, advisor):
        self.name = name
        self.sid = sid
        self.advisor = advisor

    def introduce(self):
        print(self.name, self.sid, self.advisor)
```

- 代码高度重复
- 维护成本极高
- 修改一处, 容易漏另一处

问题的本质

- 两个类:
 - 有共同的数据
 - 有共同的行为
- 但又在某些地方略有不同

这正是“继承”要解决的问题

两种常见的类关系

■ is-a 关系 (继承)

- 研究生是学生
- GraduateStudent **is-a** Student

■ has-a 关系 (组合)

- 学生有成绩单
- Student **has-a** Transcript

只有 is-a 才适合继承

定义父类 Student

```
class Student:
    def __init__(self, name, sid):
        self.name = name
        self.sid = sid

    def introduce(self):
        print(self.name, self.sid)
```

■ 父类负责:

- 通用属性
- 通用行为

子类如何继承父类

```
class GraduateStudent(Student):  
    pass
```

- 括号中的类: 父类
- 子类自动获得父类的:
 - 属性
 - 方法

继承带来的“免费能力”

```
g = GraduateStudent("Alice", "2023001")  
g.introduce()
```

- 虽然子类中没写 `introduce`
- 但可以直接调用

子类 = 父类能力 + 自己的扩展

子类增加自己的属性

```
class GraduateStudent(Student):  
    def __init__(self, name, sid, advisor)  
        :  
        self.name = name  
        self.sid = sid  
        self.advisor = advisor
```

■ 问题:

- 父类的初始化逻辑被重复写了一遍

重写父类方法

```
class GraduateStudent(Student):  
    def introduce(self):  
        print(self.name, self.sid, self.  
              advisor)
```

■ 子类方法会:

- 覆盖父类同名方法

■ 这称为 方法重写 (override)

使用 super() 复用父类逻辑

```
class GraduateStudent(Student):  
    def __init__(self, name, sid, advisor)  
        :  
        super().__init__(name, sid)  
        self.advisor = advisor
```

■ super():

- 调用父类的方法
- 保持父类逻辑一致

super() 的直观理解

- super 不是“父类对象”
- 而是:
 - 当前类在继承链中的下一站

`super().__init__` $\Rightarrow$ `Student.__init__`

总结 1: 继承在干什么

- 继承用于描述 is-a 关系
- 父类定义共性
- 子类扩展或修改行为
- `super()` 保证逻辑一致

好的继承让系统“自然生长”

本讲的主要内容

1 1. Python 类的继承

■ 1.1 类的继承基础知识

■ 1.2 继承示例: Vector 类与子类

2 2. 案例学习

父类: 抽象意义上的 Vector

```
class Vector:
    def __init__(self, data):
        self.data = list(data)
        self.dim = len(self.data)
    def norm(self):
        return sum(x*x for x in self.data)
        ** 0.5
```

- 描述的是: “任意维向量”
- 不关心具体是 2D、3D 还是别的

二维向量的特殊需求

- 维数固定为 2
- 经常使用:
 - x, y 坐标
 - 平面几何
- 但本质仍然是向量

二维向量 is-a 向量

子类: Vector2D

```
class Vector2D(Vector):  
    def __init__(self, x, y):  
        super().__init__([x, y])
```

■ 子类:

- 复用了父类的数据结构
- 只负责提供更友好的接口

继承带来的直接收益

```
v = Vector2D(3, 4)
print(v.norm())
```

■ Vector2D 中:

- 没有写 norm()
- 但可以直接用

子类的专属接口

```
class Vector2D(Vector):  
    def __init__(self, x, y):  
        super().__init__([x, y])  
  
    @property  
    def x(self):  
        return self.data[0]  
  
    @property  
    def y(self):  
        return self.data[1]
```

- 父类不知道 x, y , 子类可以非常自然地补充

子类: Vector3D

```
class Vector3D(Vector):  
    def __init__(self, x, y, z):  
        super().__init__([x, y, z])
```

■ 与 Vector2D:

- 结构相同
- 语义不同

什么是稀疏向量?

- 大多数分量为 0
- 只存非零元素
- 数据结构完全不同

但语义仍然是“向量”

子类: SparseVector

```
class SparseVector(Vector):  
    def __init__(self, dim, values)  
        :  
        self.dim = dim  
        self.values = dict(values)
```

- 不再使用 list 存数据
- 使用 dict 存非零项

SparseVector 重写 norm()

```
class SparseVector(Vector):  
    def norm(self):  
        return sum(v*v for v in self.  
                    values.values()) ** 0.5
```

- 方法名相同
- 行为不同

同一个接口，不同对象

```
vectors = [  
    Vector([1, 2, 3]),  
    Vector2D(3, 4),  
    SparseVector(1000, {3: 4, 10: 5}) ]  
  
for v in vectors:  
    print(v.norm())
```

这正是“多态”的核心思想

继承的正确使用方式

- 继承 = 语义层级
- 父类定义 “是什么”
- 子类定义 “有什么不同”
- 接口一致，内部可变

从“能用”到“好用”：工程视角下的类

- 语法正确 $\neq$ 设计合理
- 工程代码强调：
 - 清晰的职责
 - 稳定的接口
 - 可维护性
- 好代码是“被模仿出来的”

接下来通过几个“值得学习的类设计实例”
来体会

实例 1: 封装状态, 避免外部破坏

```
class Counter:
    def __init__(self):
        self._value = 0

    def increment(self):
        self._value += 1

    @property
    def value(self):
        return self._value
```

- 状态由对象自己维护; 外部不能随意修改; 接口简单、行为可控

实例 2: 像属性一样使用“计算结果”

```
class Vector:
    def __init__(self, data):
        self.data = list(data)

    @property
    def norm(self):
        return sum(x*x for x in self.data)
        ** 0.5
```

- `v.norm` 而不是 `v.norm()`
- 读起来更像数学表达式
- 实现细节被隐藏

实例 3: 限制用户的“操作自由度”

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self._score = score

    @property
    def score(self):
        return self._score
```

- 成绩可读，但不可随意改
- 防止非法状态出现
- 类对自身状态负责

实例 4: 继承中“只改一点点”

```
class LoggedVector(Vector):  
    def norm(self):  
        print("Computing norm")  
        return super().norm()
```

- 行为语义保持一致
- 在父类基础上增强
- 不破坏原有设计

一个值得模仿的类结构

```
class Vector:
    def __init__(self, data):
        self._data = list(data)

    def __len__(self):
        return len(self._data)

    def __add__(self, other):
        return Vector(a + b for a, b in zip(self._data,
                                             other._data))

    @property
    def norm(self):
        return sum(x*x for x in self._data) ** 0.5
```

总结

- 状态集中在对象内部
- 接口尽量少而清晰
- 使用 @property 提升可读性
- override 时尊重父类语义
- 让“错误用法”尽量写不出来

这就是从“会写代码”走向“工程代码”的第一步

Thanks!